ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN 1

Ma Trận và Cơ Sở của Tính Toán Khoa Học

Môn học: Toán Ứng Dụng và Thống Kê
Lớp: 24CTT3

Nhóm 12 (CQ2024-3)

Đinh Đức Hiếu	24120002
Liên Trung Hiếu	24120049
Trương Đình Nhật Huy	24120064
Nguyễn Ngọc Quang	24120127
Đặng Quang Tiến	24120149

Giảng viên hướng dẫn

ThS. Võ Nam Thục Đoan
ThS. Lê Nhựt Nam

Thành phố Hồ Chí Minh, ngày 20 tháng 4 năm 2026

Mục lục

1 Phân công công việc và đánh giá **2**
2 Giới thiệu đồ án **3**
2.1 Mở đầu 3
2.2 Cấu trúc thư mục 3
3 Phép khử Gauss và ứng dụng **4**
3.1 Cài đặt thuật toán 4
3.2 Kiểm chứng và kết quả 6
4 Phân rã ma trận và trực quan hóa với Manim **11**
4.1 Chéo hóa ma trận (Diagonalization) 11
4.2 Phân rã ma trận 11
4.3 Thư viện trực quan hóa Manim 12
5 Giải hệ phương trình và phân tích hiệu năng **13**
5.1 Mục tiêu 13
5.2 Thiết kế thực nghiệm và tiêu chí đánh giá 13
5.3 Kết quả và thảo luận 15
5.4 Kết luận của phần thực nghiệm 17
6 Kết luận **19**
6.1 Tóm tắt kết quả đạt được 19
6.2 Khó khăn gặp phải 19
6.3 Bài học rút ra 19
7 Phụ lục **20**
7.1 Tổng hợp các trường hợp kiểm chứng 20
7.2 Thiết lập benchmark 20
7.3 Bảng số liệu benchmark 20
8 Tài liệu tham khảo **22**

1 | Phân công công việc và đánh giá

Bảng 1 : Phân công công việc và đánh giá mức độ đóng góp của các thành viên

STT	MSSV	Họ và tên	Vai trò	Nhiệm vụ	Đánh giá
1	24120002	Đinh Đức Hiếu	Thành viên	Thực hiện phần 1 (Phép khử Gauss và các ứng dụng)	100%
2	24120049	Liên Trung Hiếu	Thành viên	Thực hiện phần 3 (Giải hệ phương trình và benchmark)	100%
3	24120064	Trương Đình Nhật Huy	Thành viên	Thực hiện phần 2 (Manim)	100%
4	24120127	Nguyễn Ngọc Quang	Nhóm trưởng	Quản lý đồ án, viết kiểm thử thực nghiệm và tổng hợp báo cáo	100%
5	24120149	Đặng Quang Tiến	Thành viên	Thực hiện phần 2 (Chéo hóa và phân rã SVD)	100%

2 | Giới thiệu đồ án

2.1 Mở đầu

Đồ án môn học này được xây dựng với mục tiêu giúp sinh viên không chỉ nắm vững các định lý toán học thuần túy, mà còn hiểu rõ cách những công thức đó vận hành bên trong kiến trúc máy tính.

Nội dung trọng tâm của đồ án được cấu trúc thành ba trục kiến thức chính:

Phần 1: Phép khử Gauss và ứng dụng. Tập trung nghiên cứu phép khử `Gauss` và chiến lược chọn phần tử chốt (`Partial Pivoting`). Đây là nền tảng quan trọng để giải quyết các bài toán kinh điển như giải hệ phương trình tuyến tính, tính định thức và tìm ma trận nghịch đảo.

Phần 2: Phân rã ma trận và trực quan hóa với Manim. Đi sâu vào cấu trúc nội tại của ma trận thông qua kỹ thuật chéo hóa và phân rã kỳ dị (`SVD`). Phân tích ma trận thành các phép biến đổi không gian (xoay, giãn, nén). Đồng thời, tích hợp thư viện `Manim` để xây dựng các hoạt cảnh trực quan, minh họa những khái niệm trừu tượng.

Phần 3: Giải hệ phương trình và phân tích hiệu năng. Thực hiện các bài đo đạc (`benchmark`) để so sánh các giải thuật trực tiếp (`Gauss`, `LU`) với phương pháp lặp (`Gauss-Seidel`). Thông qua việc khảo sát trên các lớp ma trận đặc biệt như `SPD` hay ma trận `Hilbert` (điều kiện kém), làm rõ mối quan hệ giữa số điều kiện (`Condition Number`) và độ tin cậy của nghiệm số.

Toàn bộ đồ án được thực hiện trên ngôn ngữ `Python`. Tự cài đặt lõi thuật toán, sau đó sử dụng các thư viện chuẩn như `NumPy`, `SciPy` để đối chiếu và kiểm chứng.

2.2 Cấu trúc thư mục

Để đảm bảo tính module và dễ quản lý, mở rộng và bảo trì, mã nguồn của đồ án được tổ chức theo cấu trúc phân cấp như sau:

```
Group_12/
├── part1/           # Thuật toán khử Gauss và ứng dụng
├── part2/           # Chéo hóa và phân rã ma trận SVD
├── part3/           # Giải hệ phương trình và Benchmark
├── report/          # Mã nguồn Typst và tài liệu báo cáo
└── requirements.txt
└── README.md
```

3 | Phép khử Gauss và ứng dụng

Phần này tập trung vào phép khử Gauss và các ứng dụng trực tiếp của nó trong đại số tuyến tính số. Thay vì chỉ sử dụng phép khử Gauss để giải hệ phương trình tuyến tính, nhóm khai thác cùng một ý tưởng biến đổi dòng để xây dựng các chức năng quan trọng khác như tính định thức, tìm ma trận nghịch đảo, xác định hạng và mô tả các cơ sở liên quan của ma trận.

Vì các bài toán này có chung nền tảng toán học nhưng khác nhau ở đầu ra cần thu được, nhóm tổ chức phần cài đặt thành các hàm riêng biệt nhằm giúp việc kiểm thử, giải thích và tái sử dụng thuật toán được rõ ràng hơn.

3.1 Cài đặt thuật toán

3.1.1 `gaussian_eliminate(A, b)`

Mục tiêu: giải hệ $Ax = b$ bằng phép khử Gauss và phân loại loại nghiệm theo 3 trường hợp: có nghiệm duy nhất, có vô số nghiệm hoặc vô nghiệm.

Đầu vào: ma trận hệ số kích thước $m \times n$ (kí hiệu: A), vector vế phải kích thước $m \times 1$ (kí hiệu: b), ngưỡng nhận diện số gần bằng 0 (kí hiệu: ε, tùy chọn với giá trị mặc định 10^{-12}).

Đầu ra: ma trận tăng cường ở dạng `REF`, một cấu trúc `solution_info` mô tả loại nghiệm và nội dung nghiệm, cùng với `swap_count` là số lần hoán đổi dòng.

Ý tưởng thuật toán: Trước hết, hệ được viết dưới dạng ma trận tăng cường $[A \mid b]$. Thuật toán thực hiện khử Gauss có `partial pivoting`: ở mỗi cột, chọn phần tử có trị tuyệt đối lớn nhất làm `pivot` để giảm rủi ro chia cho số quá nhỏ. Sau khi khử về dạng `REF`, hệ được phân loại như sau:

- Nếu xuất hiện hàng $[0 \ \dots \ 0 \mid c]$ với $c \neq 0$, hệ vô nghiệm;
- Nếu số cột `pivot` bằng số ẩn, hệ có nghiệm duy nhất;
- Nếu còn cột tự do, hệ có vô số nghiệm.

Cách cài đặt: Trong code, ma trận tăng cường được tạo bằng `_augment`. Bước khử tiến được thực hiện trong `_forward_elimination_ref`, đồng thời theo dõi `pivot_columns`, `swap_count` và cảnh báo nếu `pivot` quá nhỏ. Từ kết quả khử, quá trình phân nhánh theo từng loại nghiệm:

- Nếu xuất hiện hàng mâu thuẫn thì trả về hệ vô nghiệm;
- Nếu hệ có nghiệm duy nhất, chương trình tách hệ tam giác trên bằng `_extract_upper_system` rồi chuyển cho `back_substitution`.
- Nếu hệ có vô số nghiệm, chương trình dùng `_rref` và `_build_general_solution_from_rref` để dựng nghiệm tổng quát.

Cách tổ chức này giúp phần cài đặt bám sát đúng ba trường hợp đã nêu trong phần ý tưởng thuật toán, đồng thời làm cho từng nhánh xử lý có thể được kiểm tra độc lập.

3.1.2 `back_substitution(U, c, eps)`

Mục tiêu: Giải hệ tam giác trên $Ux = c$ trong trường hợp hệ có nghiệm duy nhất.

Đầu vào: Ma trận tam giác trên U, vector vế phải c, và ngưỡng ε.

Đầu ra: Vector nghiệm x.

Ý tưởng thuật toán: Thực hiện thế ngược từ hàng cuối lên hàng đầu. Ở mỗi bước, một ẩn được tính dựa trên các ẩn đã biết trước đó.

Cách cài đặt: Hàm duyệt chỉ số hàng từ cuối về đầu, tính phần tổng các hạng tử đã biết rồi suy ra giá trị của ẩn hiện tại. Nếu phần tử trên đường chéo chính quá nhỏ hoặc bằng 0, hàm báo lỗi vì không thể tiếp tục thế ngược.

3.1.3 `determinant(A, eps)`

Mục tiêu: Tính định thức của ma trận vuông.

Đầu vào: Ma trận vuông `A` và ngưỡng `eps`.

Đầu ra: Giá trị định thức của ma trận.

Ý tưởng thuật toán: Đưa ma trận về dạng tam giác trên bằng phép khử Gauss. Khi đó, định thức được tính bằng tích các phần tử trên đường chéo. Nếu có đổi dòng, dấu của định thức thay đổi theo số lần hoán đổi.

Cách cài đặt: Hàm tái sử dụng `_forward_elimination_ref` để nhận ma trận sau khử và `swap_count`. Sau đó tính tích các phần tử đường chéo và nhân với dấu $(-1)^s$. Nếu đầu vào không phải ma trận vuông A, hàm báo lỗi.

3.1.4 `inverse(A, eps)`

Mục tiêu: Tính ma trận nghịch đảo A^{-1}.

Đầu vào: Ma trận vuông `A` và ngưỡng `eps`.

Đầu ra: Ma trận nghịch đảo của A.

Ý tưởng thuật toán: Dùng phương pháp `Gauss-Jordan` trên ma trận ghép $[A \mid I]$. Nếu đưa được vế trái về ma trận đơn vị I, thì vế phải chính là A^{-1}.

Cách cài đặt: Hàm ghép A với ma trận đơn vị cùng kích thước. Với mỗi cột, chương trình chọn `pivot` phù hợp, đổi dòng nếu cần, chuẩn hóa dòng `pivot` rồi khử tất cả các dòng còn lại. Sau khi hoàn tất, phần bên phải của ma trận ghép được tách ra làm ma trận nghịch đảo. Nếu không chọn được `pivot` hợp lệ, hàm báo lỗi do ma trận suy biến.

3.1.5 `rank_and_basis(A, eps)`

Mục tiêu: Tính hạng của ma trận và xác định các cơ sở liên quan.

Đầu vào: Ma trận A và ngưỡng ε.

Đầu ra: Một cấu trúc dữ liệu chứa `rank`, `pivot_columns`, `free_columns`, `column_space_basis`, `row_space_basis` và `null_space_basis`.

Từ dạng `RREF`, số cột `pivot` cho biết hạng ma trận. Các cột `pivot` của ma trận gốc tạo thành cơ sở không gian cột; các dòng khác 0 của `RREF` tạo thành cơ sở không gian dòng; các cột tự do được dùng để dựng cơ sở không gian nghiệm (`Nullspace`).

Cách cài đặt: Hàm gọi `_rref` để lấy ma trận rút gọn và danh sách `pivot_columns`. Từ đó, chương trình suy ra `free_columns` và lần lượt dựng các basis tương ứng. Đây là bước chuyển trực tiếp từ khái niệm toán học sang cấu trúc dữ liệu có thể dùng tiếp trong code.

3.1.6 `verify_solution(A, x, b, tol)`

Mục tiêu: Kiểm chứng nghiệm hoặc thông tin nghiệm do phần cài đặt chính trả về.

Đầu vào: Ma trận A, nghiệm hoặc `solution_info`, vector b, và ngưỡng τ.

Đầu ra: Một cấu trúc chứa `is_close`, `residual_norm`, `Relative_residual` và nghiệm tham chiếu từ `NumPy`.

Ý tưởng thuật toán: Không tham gia vào phần tính toán lõi. Hàm này chỉ dùng để đối chiếu kết quả và đo mức độ sai số thông qua `residual`.

Cách cài đặt: Nếu đầu vào là nghiệm duy nhất hoặc nghiệm riêng của hệ vô số nghiệm, hàm chuyển sang mảng `NumPy` để kiểm tra $Ax \approx b$. Nếu đầu vào thuộc nhánh vô nghiệm, hàm báo lỗi thay vì cố kiểm tra một nghiệm không tồn tại.

3.2 Kiểm chứng và kết quả

Sau khi hoàn thành phần cài đặt, các case tiêu biểu được sử dụng để kiểm tra thuật toán. Mục tiêu của phần này không chỉ là xác nhận chương trình chạy đúng, mà còn làm rõ ý nghĩa toán học của từng loại kết quả và đối chiếu với hành vi mong đợi của thuật toán.

3.2.1 Pivot ban đầu gần bằng 0

Trong trường hợp này, đồ án sử dụng một ma trận 3×3 có tính chất đặc biệt: phần tử đầu tiên A_{11} cực nhỏ (10^{-10}). Nếu không có `partial pivoting`, thuật toán sẽ chia cho một số gần bằng 0, dẫn đến sai số làm tròn cực lớn.

Xét hệ phương trình tuyến tính $A_1 x = b_1$, trong đó:

$$A_1 = \begin{pmatrix} 10^{-10} & 2.34567 & -1.23456 \\ 3.5 & -4.2 & 1.125 \\ 2.25 & 1.33333 & 5.75 \end{pmatrix}, b_1 = \begin{pmatrix} 1.2345 \\ -2.3456 \\ 3.4567 \end{pmatrix}. \tag{1}$$

Phần tử ở vị trí đầu tiên của A_1 là 10^{-10}, có độ lớn rất nhỏ. Vì vậy, nếu chọn trực tiếp phần tử này làm pivot thì phép chia trong quá trình khử có thể gây sai số lớn. Khi dùng `partial pivoting`, thuật toán đổi dòng để chọn pivot có trị tuyệt đối lớn hơn.

Ma trận tăng cường sau khử Gauss thu được:

$$M_1^{\mathrm{REF}} = \begin{pmatrix} 3.5000 & -4.2000 & 1.1250 & -2.3456 \\ 0.0000 & 4.0333 & 5.0268 & 4.9646 \\ 0.0000 & 0.0000 & -4.1580 & -1.6528 \end{pmatrix}. \tag{2}$$

Các cột pivot là:

$$P_1 = [0, 1, 2], \;\; s_1 = 2, \tag{3}$$

trong đó s_1 là số lần hoán đổi dòng. Vì cả ba cột ẩn đều là cột pivot, hệ có nghiệm duy nhất. Nghiệm trả về bởi `gaussian_eliminate` là:

$$x_1 = \begin{pmatrix} 0.0847 \\ 0.7355 \\ 0.3975 \end{pmatrix}. \tag{4}$$

Khi tách hệ tam giác trên từ M_1^{REF} và áp dụng thế ngược, ta cũng thu được:

$$x_1^{\text{back}} = \begin{pmatrix} 0.0847 \\ 0.7355 \\ 0.3975 \end{pmatrix}. \tag{5}$$

Định thức và ma trận nghịch đảo của A_1 là:

$$\det(A_1) = -58.6970. \tag{6}$$

$$A_1^{-1} = \begin{pmatrix} 0.4370 & 0.2578 & 0.0434 \\ 0.2997 & -0.0473 & 0.0736 \\ -0.2405 & -0.0899 & 0.1399 \end{pmatrix}. \tag{7}$$

Kết quả kiểm chứng nghiệm:

$$\|A_1 x_1 - b_1\|_2 = 0.0000, \quad \frac{\|A_1 x_1 - b_1\|_2}{\|b_1\|_2} = 0.0000. \tag{8}$$

Nghiệm tham chiếu từ `NumPy` là:

$$x_1^{\text{numpy}} = \begin{pmatrix} 0.0847 \\ 0.7355 \\ 0.3975 \end{pmatrix}. \tag{9}$$

- *Kết quả thực nghiệm*: Như vậy, nghiệm từ thuật toán, nghiệm từ thế ngược và nghiệm tham chiếu trùng nhau ở bốn chữ số thập phân. Kết quả này cho thấy việc đổi dòng đã xử lý đúng trường hợp pivot ban đầu gần bằng 0.

3.2.2 Hệ có vô số nghiệm

Khả năng phân loại hệ được kiểm chứng bằng một ma trận phụ thuộc tuyến tính kích thước 3×4. Ở trường hợp này, mục tiêu không phải là một nghiệm duy nhất mà là cấu trúc của không gian nghiệm.

Xét hệ $A_2 x = b_2$ với A_2 là ma trận 3×4:

$$A_2 = \begin{pmatrix} 1.2 & 2.4 & -0.6 & 3.0 \\ 2.4 & 4.8 & -1.2 & 6.0 \\ 0.6 & 1.2 & -0.3 & 1.5 \end{pmatrix}, b_2 = \begin{pmatrix} 3.6 \\ 7.2 \\ 1.8 \end{pmatrix}. \tag{10}$$

Các dòng của A_2 phụ thuộc tuyến tính. Cụ thể, dòng thứ hai bằng 2 lần dòng thứ nhất và dòng thứ ba bằng 0.5 lần dòng thứ nhất. Sau khi khử Gauss, ma trận tăng cường có dạng:

$$M_2^{\text{REF}} = \begin{pmatrix} 2.4000 & 4.8000 & -1.2000 & 6.0000 & 7.2000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \end{pmatrix}. \tag{11}$$

Do chỉ có một cột pivot nên:

$$\operatorname{rank}(A_2) = 1,\ \ P_2 = [0],\ \ F_2 = [1, 2, 3]. \tag{12}$$

Trong đó P_2 là tập cột pivot và F_2 là tập cột tự do. Vì số ẩn là 4 nhưng hạng của ma trận chỉ bằng 1, hệ có vô số nghiệm.

Nghiệm tổng quát do chương trình dựng từ dạng rút gọn theo dòng là:

$$x = \begin{pmatrix} 3.0000 \\ 0.0000 \\ 0.0000 \\ 0.0000 \end{pmatrix} + t_1 \begin{pmatrix} -2.0000 \\ 1.0000 \\ 0.0000 \\ 0.0000 \end{pmatrix} + t_2 \begin{pmatrix} 0.5000 \\ 0.0000 \\ 1.0000 \\ 0.0000 \end{pmatrix} + t_3 \begin{pmatrix} -2.5000 \\ 0.0000 \\ 0.0000 \\ 1.0000 \end{pmatrix},\ \ t_1, t_2, t_3 \in \mathbb{R}. \tag{13}$$

Từ đó, một nghiệm riêng là:

$$x_p = \begin{pmatrix} 3.0000 \\ 0.0000 \\ 0.0000 \\ 0.0000 \end{pmatrix}. \tag{14}$$

Cơ sở của không gian nghiệm thuần nhất $N(A_2)$ gồm ba vector:

$$N(A_2) = \operatorname{span}\left(\begin{pmatrix} -2.0000 \\ 1.0000 \\ 0.0000 \\ 0.0000 \end{pmatrix}, \begin{pmatrix} 0.5000 \\ 0.0000 \\ 1.0000 \\ 0.0000 \end{pmatrix}, \begin{pmatrix} -2.5000 \\ 0.0000 \\ 0.0000 \\ 1.0000 \end{pmatrix} \right). \tag{15}$$

Kết quả `rank_and_basis` cũng cho:

$$\operatorname{Col}(A_2) = \operatorname{span}\left(\begin{pmatrix} 1.2000 \\ 2.4000 \\ 0.6000 \end{pmatrix} \right). \tag{16}$$

$$\operatorname{Row}(A_2) = \operatorname{span}((1.0000\ \ 2.0000\ \ -0.5000\ \ 2.5000)). \tag{17}$$

Kết quả kiểm chứng nghiệm riêng:

$$\|A_2 x_p - b_2\|_2 = 0.0000,\ \ \frac{\|A_2 x_p - b_2\|_2}{\|b_2\|_2} = 0.0000. \tag{18}$$

Nghiệm tham chiếu từ `NumPy` là một nghiệm cụ thể khác của cùng hệ:

$$x_2^{(\text{numpy})} = \begin{pmatrix} 0.2609 \\ 0.5217 \\ -0.1304 \\ 0.6522 \end{pmatrix}. \tag{19}$$

- *Kết quả thực nghiệm*: Sự khác nhau giữa x_p và $x_2^{(\text{numpy})}$ không mâu thuẫn, vì vậy hệ có vô số nghiệm.

3.2.3 Hệ vô nghiệm

Để kiểm tra nhánh xử lý mâu thuẫn, một hệ phương trình không tương thích được đưa vào thực nghiệm.

Xét hệ $A_3x = b_3$:

$$A_3 = \begin{pmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 1 & -1 & 0 \end{pmatrix}, b_3 = \begin{pmatrix} 3 \\ 7 \\ 1 \end{pmatrix}. \tag{20}$$

Sau khi khử Gauss, ma trận tăng cường thu được:

$$M_3^{\text{REF}} = \begin{pmatrix} 2.0000 & 2.0000 & 2.0000 & 7.0000 \\ 0.0000 & -2.0000 & -1.0000 & -2.5000 \\ 0.0000 & 0.0000 & 0.0000 & -0.5000 \end{pmatrix}. \tag{21}$$

Dòng cuối của ma trận trên tương ứng với phương trình:

$$0x_1 + 0x_2 + 0x_3 = -0.5000. \tag{22}$$

Đây là mệnh đề sai, vì vế trái luôn bằng 0 nhưng vế phải khác 0. Do đó:

$$S_3 = \emptyset. \tag{23}$$

Chương trình trả về loại nghiệm `inconsistent`, với các cột pivot:

$$P_3 = [0, 1], \;\; s_3 = 2. \tag{24}$$

- *Kết quả thực nghiệm*: Sau bước khử tiến, thuật toán phát hiện hàng có dạng $[0 \; \dots \; 0 \mid c]$ với $c \neq 0$. Chương trình dừng lại ngay lập tức và trả về `type: inconsistent`. Điều này bảo vệ hệ thống khỏi việc cố gắng tính toán trên một tập nghiệm rỗng. Trường hợp này không thực hiện thế ngược, không tính nghịch đảo và không kiểm chứng residual của nghiệm, vì tập nghiệm rỗng.

3.2.4 Hệ gần suy biến và cảnh báo `ill-conditioned`

Trường hợp cuối cùng sử dụng ma trận có các dòng gần như song song (ví dụ: các hệ số lệch nhau 10^{-10}).

Xét hệ $A_4x = b_4$:

$$A_4 = \begin{pmatrix} 1.0 & 1.0 \\ 1.0 & 1.0 + 10^{-10} \end{pmatrix}, b_4 = \begin{pmatrix} 2.0 \\ 2.0 + 10^{-10} \end{pmatrix}. \tag{25}$$

Hai dòng của A_4 gần như trùng nhau, chỉ khác nhau ở lượng rất nhỏ 10^{-10}. Sau bước khử, nếu chưa làm tròn, dòng thứ hai có pivot cỡ 10^{-10}:

$$M_4^{\text{REF}} \approx \begin{pmatrix} 1.0000 & 1.0000 & 2.0000 \\ 0.0000 & 10^{-10} & 10^{-10} \end{pmatrix}. \tag{26}$$

Khi in ra với bốn chữ số thập phân, dòng thứ hai được hiển thị như sau:

$$(0.0000 \;\; 0.0000 \;\; 0.0000). \tag{27}$$

Tuy nhiên, pivot thứ hai không bằng 0 tuyệt đối mà chỉ rất nhỏ. Vì vậy, chương trình vẫn phân loại hệ là có nghiệm duy nhất nhưng đồng thời phát sinh cảnh báo pivot gần bằng 0.

Nghiệm trả về là:

$$x_4 = \begin{pmatrix} 1.0000 \\ 1.0000 \end{pmatrix}, \ P_4 = [0, 1], \ s_4 = 0. \tag{28}$$

Kết quả kiểm chứng:

$$\|A_4 x_4 - b_4\|_2 = 0.0000, \ \frac{\|A_4 x_4 - b_4\|_2}{\|b_4\|_2} = 0.0000. \tag{29}$$

Nghiệm tham chiếu từ `NumPy` là:

$$x_4^{(\text{numpy})} = \begin{pmatrix} 1.0000 \\ 1.0000 \end{pmatrix}. \tag{30}$$

- *Kết quả thực nghiệm*: Thuật toán vẫn giải ra nghiệm nhưng đồng thời đưa ra cảnh báo `pivot is near zero`; `the system may be ill-conditioned`. Cảnh báo này cực kỳ quan trọng trong tính toán khoa học, giúp người dùng nhận thức được độ tin cậy của kết quả khi làm việc với ma trận nhạy cảm về số học.

4 | Phân rã ma trận và trực quan hóa với Manim

4.1 Chéo hóa ma trận (Diagonalization)

Thay vì thiết lập chéo hóa thành một mảng thuật toán phức tạp, phần cài đặt được thiết kế gọn gàng dựa trên ma trận demo. Cụ thể, đầu vào hàm sẽ tiếp nhận một nhóm ma trận kiểm thử đại diện và thực hiện chéo hóa nguyên bản từ đầu, bao gồm các công đoạn thuật toán số học chi tiết sau đối với nhóm ma trận nhỏ:

- *Thiết lập đa thức đặc trưng (Thuật toán `Faddeev-LeVerrier`)*: Hàm trích xuất chuỗi hệ số của đa thức $c_n, c_{n-1}, \ldots$ bằng kỹ thuật biến đổi trên vết (Trace) đệ quy của ma trận, né tránh được độ phức tạp bù trừ định thức cồng kềnh.
- *Dò tìm Trị riêng (Phương pháp lặp `Durand-Kerner`)*: Thuật toán lặp đồng thời `Durand-Kerner` được triển khai để dò tìm trên không gian nghiệm phức, giúp giải chính xác toàn bộ rễ của đa thức (tức là Trị riêng).
- *Dựng không gian Vector riêng*: Tái sử dụng thuật toán khử Gauss tìm hạng từ Mục 3, không gian nghiệm `Nullspace` của hệ $(A - \lambda I)$ được thiết lập tính toán. Các vector tìm được sẽ tiếp tục qua rây lọc *Trực chuẩn hóa `Gram-Schmidt`* nhằm bảo toàn tuyệt đối không gian nền cơ sở.

Từ các nguyên liệu trên, chương trình sẽ kiểm tra tính chéo hóa khả thi của ma trận (có đủ n vector cơ sở độc lập tuyến tính) nhằm hoàn thành bài toán tổng kết định dạng biểu diễn:

$$A = PDP^{-1} \tag{31}$$

4.2 Phân rã ma trận

Phân rã kỳ dị (Singular Value Decomposition - `SVD`) được chọn làm phương pháp cốt lõi để nghiên cứu.

Lý do lựa chọn: Khác với một số cách phân rã bị giới hạn bởi tính đối xứng hay dạng vuông của ma trận, `SVD` linh hoạt xử lý mọi kích cỡ ma trận $m \times n$. Phân rã này phân tích mọi cấu trúc dữ liệu thành các phép biến đổi không gian trực quan (rotate-scale-rotate), tạo nền tảng vững chắc cho nhiều ứng dụng khoa học lớn như nén dữ liệu.

Thành phần cấu trúc và ý nghĩa: Phân rã biến đổi ma trận gốc đầu vào A thành hệ đầu ra 3 thành phần:

$$A = U\Sigma V^T \tag{32}$$

- *Đầu vào*: Ma trận A bất kỳ $(m \times n)$.
- *Đầu ra*: Ma trận chéo Σ chứa tập hợp *trị kỳ dị (singular values)* đại diện cho mức độ co giãn. Bộ hệ cơ sở trực chuẩn U (Left singular vectors) mô tả không gian đích và V (Right singular vectors) đóng vai trò căn phối dữ liệu nguồn. Việc thu nhận phân tích giúp giải thích chính xác hệ thống tọa độ đang phóng to và dịch chuyển theo góc uốn nào.

Cột thư viện ma trận U được đối xứng qua biểu thức $u_i = \frac{A \cdot v_i}{\sigma_i}$. Đặc biệt, tính ổn định luôn được đảm bảo: mỗi khi A thất bại thiết lập trực giao do lỗi mất hạng (`Nullspace`), thuật toán thực hiện chèn trực giao hóa `Gram-Schmidt` kèm hệ cơ sở chuẩn để thu được toàn bộ các cột, đảm bảo cấu trúc U giữ nguyên vẹn.

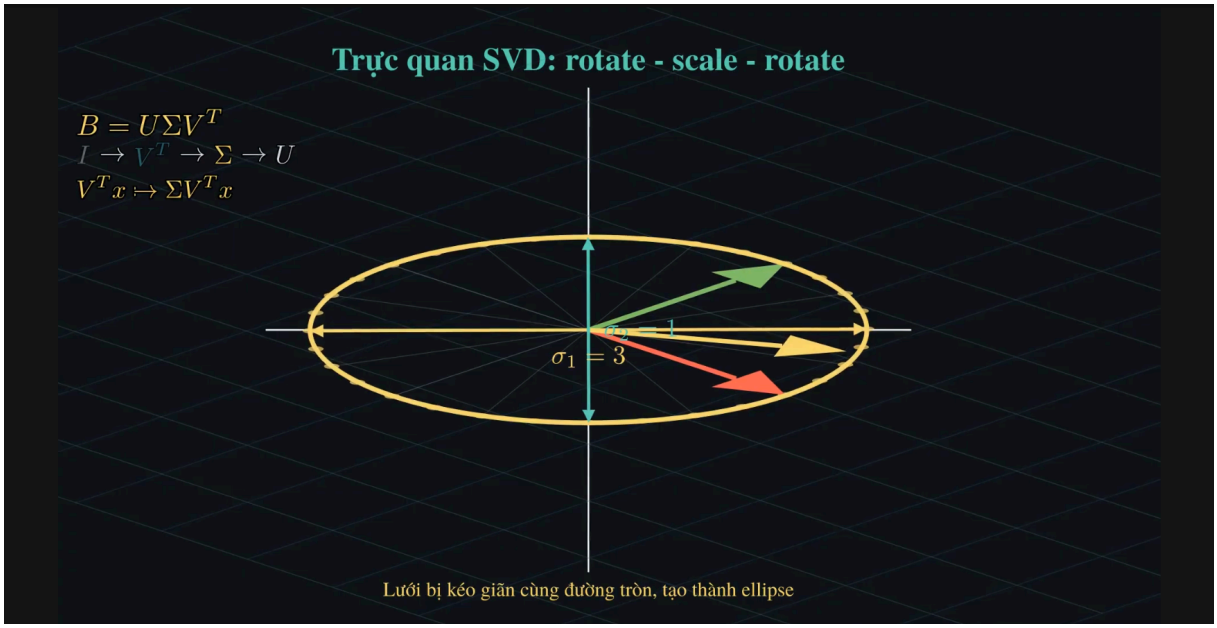
4.3 Thư viện trực quan hóa Manim

`Manim` là một thư viện Python được thiết kế để tạo ra các hình ảnh động toán học chất lượng cao. Trong đồ án này, `Manim` được sử dụng để trực quan hóa các khái niệm trừu tượng của đại số tuyến tính, giúp dễ dàng nắm bắt các phép biến đổi ma trận, sự thay đổi cơ sở và các tính chất của trị riêng, vector riêng.

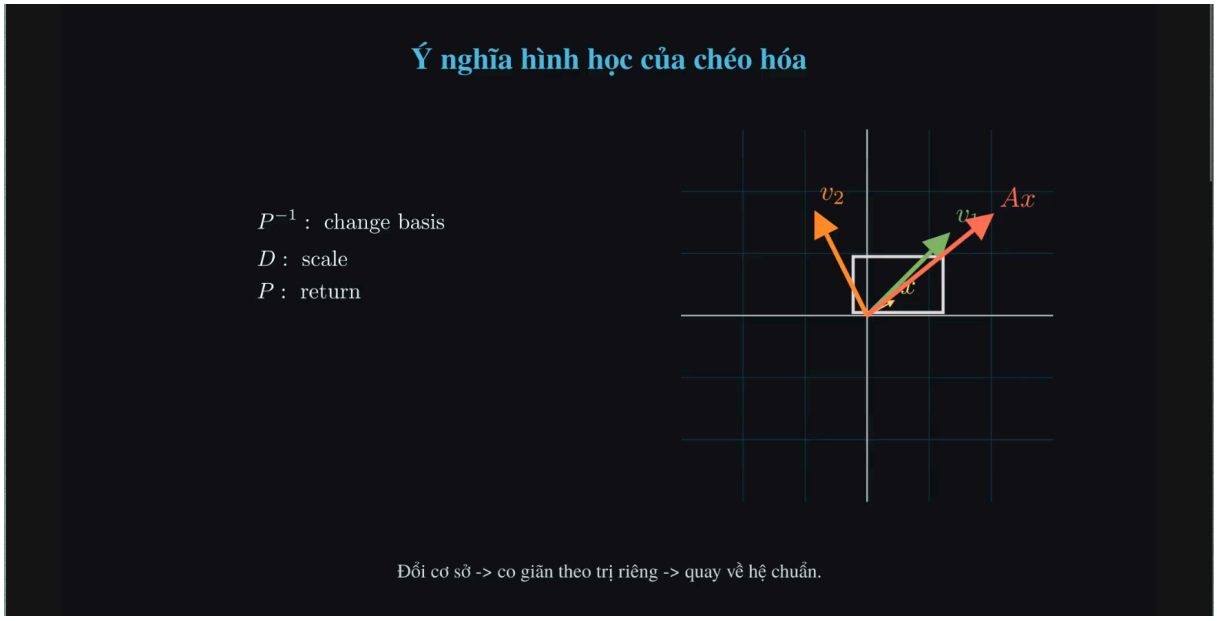
Nội dung video xoay quanh giới thiệu lý thuyết, mô phỏng thuật toán cho ma trận A cụ thể:

- Mô phỏng quá trình chéo hóa ma trận $A = PDP^{-1}$
- Trực quan quá trình phân rã SVD. Cụ thể là trục quan phép biến đổi hình học rotate-scale-rotate trên hình tròn đơn vị.

Link video: https://youtu.be/sWpLmDTU0R0



Hình 1 : Minh họa hệ vector thay đổi cơ sở và dãn không gian theo các trị kỳ dị (SVD)



Hình 2 : Tiến trình phát hiện và tính toán các đặc trưng giá trị riêng dựa trên sự bảo toàn phương hướng

5 | Giải hệ phương trình và phân tích hiệu năng

5.1 Mục tiêu

Tập trung vào việc nối từ cơ sở lý thuyết của đại số tuyến tính số đến hành vi thực tế của thuật toán khi chạy trên máy tính. Mục tiêu là đánh giá và so sánh ba phương pháp giải hệ phương trình tuyến tính gồm *khử Gauss có* `partial pivoting`, *phân rã* `LU` *có* `partial pivoting`, và *phương pháp lặp* `Gauss-Seidel` trên ba phương diện chính:

- *Chi phí tính toán*: khảo sát thời gian thực thi khi kích thước ma trận tăng dần, từ đó đối chiếu với độ phức tạp lý thuyết của từng phương pháp.
- *Độ chính xác của nghiệm*: đo mức độ sai lệch của nghiệm tính được thông qua *residual error* và *solution error*.
- *Độ ổn định số học*: quan sát phản ứng của thuật toán trên các lớp ma trận có bản chất rất khác nhau, đặc biệt là ma trận điều kiện tốt và điều kiện kém.

Thông qua phần này, đồ án hướng tới trả lời một câu hỏi thực tiễn hơn là chỉ mô tả thuật toán: *khi áp dụng vào dữ liệu cụ thể, phương pháp nào phù hợp hơn, nghiệm nào đáng tin cậy hơn, và chi phí tính toán thay đổi như thế nào khi kích thước bài toán tăng lên*.

5.2 Thiết kế thực nghiệm và tiêu chí đánh giá

Để việc so sánh phản ánh đúng bản chất của từng phương pháp, hai nhóm thực nghiệm bổ sung cho nhau đã được xây dựng.

5.2.1 Nhóm thực nghiệm A: đo thời gian thực thi và sai số theo kích thước

Ở nhóm thực nghiệm thứ nhất, các ma trận vuông *chéo trội nghiêm ngặt* được sinh ra với kích thước tăng dần

$$n \in \{50, 100, 200, 500, 1000\}. \tag{33}$$

Việc chọn ma trận chéo trội nhằm bảo đảm phương pháp `Gauss-Seidel` có điều kiện hội tụ rõ ràng, đồng thời tạo ra một điều kiện thực nghiệm tương đồng để so sánh với hai phương pháp trực tiếp là Gauss và `LU`.

Với mỗi kích thước n, nghiệm chuẩn được chọn là

$$x_{\text{true}} = (1, 1, ..., 1)^T \tag{34}$$

và dựng vế phải theo công thức

$$b = Ax_{\text{true}} \tag{35}$$

Cách làm này cho phép kiểm tra trực tiếp chất lượng nghiệm trả về, vì nghiệm đúng đã được biết trước.

Mỗi thuật toán được chạy *5 lần* trên cùng một kích thước để lấy *thời gian thực thi trung bình*. Cách lặp lại nhiều lần giúp giảm ảnh hưởng của dao động môi trường thực thi và làm cho số liệu benchmark ổn định hơn.

5.2.2 Nhóm thực nghiệm B: phân tích ổn định số học

Ở nhóm thực nghiệm thứ hai, mục tiêu không chỉ là tốc độ mà còn kiểm thử độ nhạy của nghiệm trước điều kiện của ma trận. Thực nghiệm sử dụng hai lớp dữ liệu:

- *Ma trận* `Hilbert` với các kích thước

$$n \in \{3, 5, 8, 10, 12, 15\}, \tag{36}$$

  đại diện cho lớp ma trận *điều kiện kém* (*ill-conditioned*), trong đó số điều kiện tăng rất nhanh theo kích thước.
- *Ma trận* `SPD` *ngẫu nhiên* với các kích thước

$$n \in \{5, 10, 20, 50, 100\}, \tag{37}$$

  đóng vai trò nhóm đối chứng có điều kiện tốt hơn, giúp làm rõ sự khác biệt về ổn định số học.

Với mỗi ma trận, quy trình tiếp tục sử dụng nghiệm chuẩn `x_true` để dựng vế phải `b = A x_true`, sau đó giải lại bằng ba phương pháp và so sánh sai số.

5.2.3 Cài đặt và nguyên tắc thực thi

Các solver trong Mục 5 được cài đặt bằng *nested list thuần* `Python`, không sử dụng `NumPy` array trong phần lõi thuật toán. `NumPy` chỉ được dùng trong các bước sinh dữ liệu, tính chuẩn, tính số điều kiện và đối chiếu kết quả sau cùng. Điều này giúp giữ đúng tinh thần của đồ án: thư viện chỉ phục vụ `benchmark` *và verification*, không thay thế cho phần cài đặt thuật toán.

Trong ba solver được so sánh, `LU` được dùng như một phương pháp phân rã phục vụ bài toán giải hệ trong `benchmark`; mục tiêu của Mục 5 là đánh giá hiệu năng của các solver tiêu biểu, chứ không lặp lại toàn bộ phạm vi lý thuyết của Mục 4.

Để đánh giá toàn diện, bốn chỉ số chính được sử dụng:

- *Average Runtime*: thời gian thực thi trung bình sau nhiều lần chạy.
- *Relative Residual Error*:

$$\frac{\|A\hat{x} - b\|_2}{\|b\|_2}, \tag{38}$$

  phản ánh mức độ thỏa mãn của nghiệm tính được đối với hệ phương trình ban đầu.
- *Solution Error*:

$$\frac{\|\hat{x} - x_{\text{true}}\|_2}{\|x_{\text{true}}\|_2}, \tag{39}$$

  phản ánh sai lệch thực sự của nghiệm tính toán so với nghiệm chuẩn.
- *Condition Number*:

$$\kappa(A), \tag{40}$$

  dùng để giải thích mối quan hệ giữa cấu trúc ma trận và mức độ khuếch đại sai số.

Toàn bộ kết quả benchmark được lưu dưới dạng `.csv`, sau đó sử dụng notebook phân tích để dựng bảng tổng hợp và biểu đồ trực quan. Các bảng số liệu đầy đủ được đưa vào phụ lục, xem Bảng 3, Bảng 5 và Bảng 6.

5.3 Kết quả và thảo luận

Từ các số liệu benchmark thu được, có thể nhận thấy sự khác biệt rõ ràng giữa ba phương pháp cả về thời gian thực thi lẫn độ ổn định số học. Các quan sát dưới đây được rút ra từ dữ liệu tổng hợp và các biểu đồ phân tích đi kèm.

5.3.1 Thời gian thực thi và chi phí tính toán

Kết quả thực nghiệm A cho thấy hai phương pháp trực tiếp là *khử Gauss* và `LU` đều có xu hướng thời gian thực thi tăng đáng kể khi kích thước ma trận tăng lên. Điều này phù hợp với phân tích lý thuyết rằng chi phí của hai phương pháp này tăng theo bậc xấp xỉ $\mathcal{O}(n^3)$.

Ở các kích thước nhỏ, chênh lệch giữa ba phương pháp chưa rõ rệt. Tuy nhiên, khi n tăng lên các mức lớn hơn như 500 hoặc 1000, thời gian của Gauss và `LU` tăng mạnh và trở thành yếu tố chi phối. Mặt khác, dựa theo bộ dữ liệu, `Gauss-Seidel` không cho thời gian thực thi thấp hơn Gauss và `LU`. Dù mỗi vòng lặp chỉ có chi phí cỡ $\mathcal{O}(n^2)$.

tổng thời gian của phương pháp lặp vẫn phụ thuộc mạnh vào số vòng lặp cần để hội tụ và vào chi phí của cài đặt list thuần. Điều này cho thấy lợi thế lý thuyết của phương pháp lặp không tự động chuyển thành lợi thế thời gian trong mọi môi trường chạy.

Kết quả trên cho ta biết rằng, khi đánh giá chi phí tính toán thì ta phải nhìn đồng thời vào cả độ phức tạp lý thuyết và chi phí thực nghiệm.

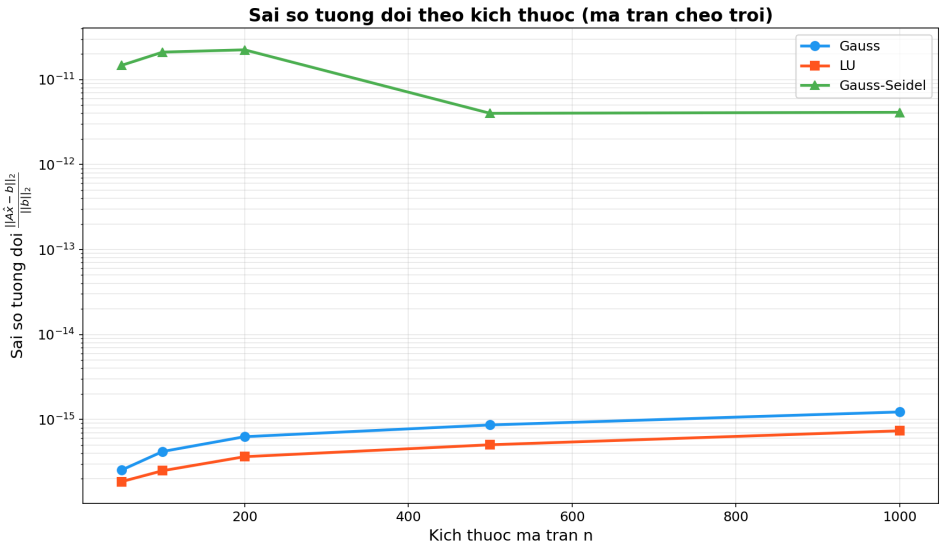
Hình 3 : Đồ thị log-log thời gian thực thi theo kích thước n

Các giá trị thời gian thực thi chi tiết theo từng kích thước được tổng hợp trong Bảng 3.

5.3.2 Sai số trên dữ liệu chéo trội

Trên nhóm ma trận chéo trội dùng trong thực nghiệm A, cả ba phương pháp đều cho *Relative residual* nhỏ. Điều này cho thấy các nghiệm tính được vẫn thỏa mãn khá tốt hệ phương trình ban đầu.

Tuy nhiên, việc residual nhỏ không đồng nghĩa với việc mọi phương pháp đều giống nhau về mặt hiệu năng. Trong bộ dữ liệu này, Gauss và `LU` vừa giữ residual rất thấp, vừa có thời gian thực thi tốt hơn `Gauss-Seidel` trong `benchmark` hiện tại. Vì vậy, dữ liệu chéo trội ở đây phù hợp để minh họa rằng việc đánh giá thuật toán không nên dựa trên một chỉ số duy nhất.



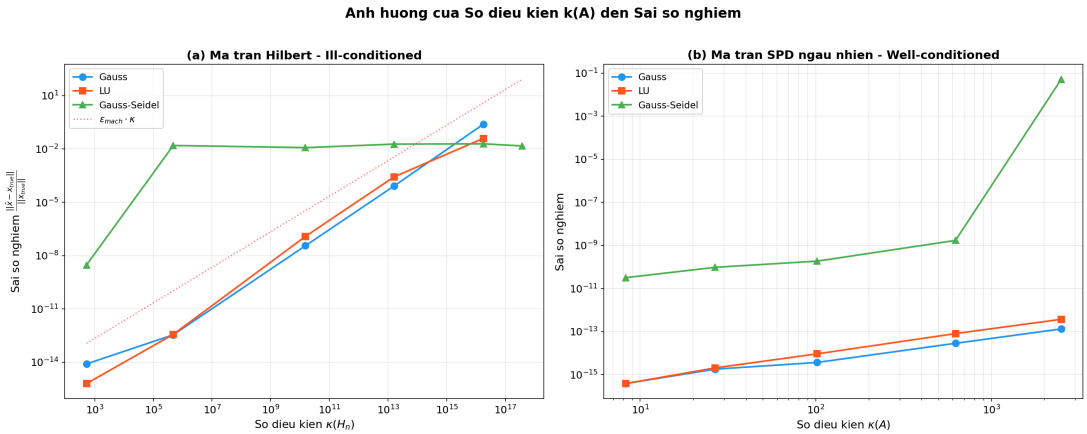
Hình 4 : Sai số tương đối theo kích thước n trên ma trận chéo trội

5.3.3 Ảnh hưởng của số điều kiện: trường hợp ma trận Hilbert và SPD

Với *ma trận* `Hilbert`, khi kích thước tăng thì số điều kiện tăng rất nhanh, khiến bài toán trở nên cực kỳ nhạy với sai số làm tròn. Kết quả thực nghiệm cho thấy *solution error* của `Gauss` và `LU` tăng rõ rệt theo `kappa(A)`, dù trong nhiều trường hợp *Relative residual* vẫn còn nhỏ. Điều này phản ánh đúng bản chất của bài toán điều kiện kém: một nghiệm có thể gần thỏa mãn phương trình `A x = b`, nhưng vẫn sai khác đáng kể so với nghiệm thực `x_true`. Nói cách khác, residual nhỏ không đồng nghĩa với nghiệm đáng tin cậy khi `kappa(A)` lớn.

Với *ma trận* `SPD` *ngẫu nhiên*, `Gauss` và `LU` duy trì sai số rất thấp trên toàn bộ dải `condition number` được khảo sát. Trong khi đó, `Gauss-Seidel` *nhạy hơn rõ rệt*: ở các mức `condition number` lớn hơn trong bộ dữ liệu hiện tại, sai số của phương pháp này tăng đáng kể so với hai phương pháp trực tiếp. Điều đó cho thấy việc một phương pháp có cơ sở hội tụ về lý thuyết không đồng nghĩa với việc nó ổn định như nhau trên mọi bộ dữ liệu hữu hạn.

Đối với `Gauss-Seidel` trên `Hilbert`, ma trận vẫn là đối xứng xác định dương nên về mặt lý thuyết phương pháp có cơ sở hội tụ. Tuy nhiên, thực nghiệm cho thấy tốc độ hội tụ và độ tin cậy của nghiệm suy giảm rõ rệt khi kích thước tăng. Trong các trường hợp khó, cần thực hiện phương pháp với nhiều vòng lặp hoặc dừng ở trạng thái mà sai số nghiệm vẫn còn lớn.

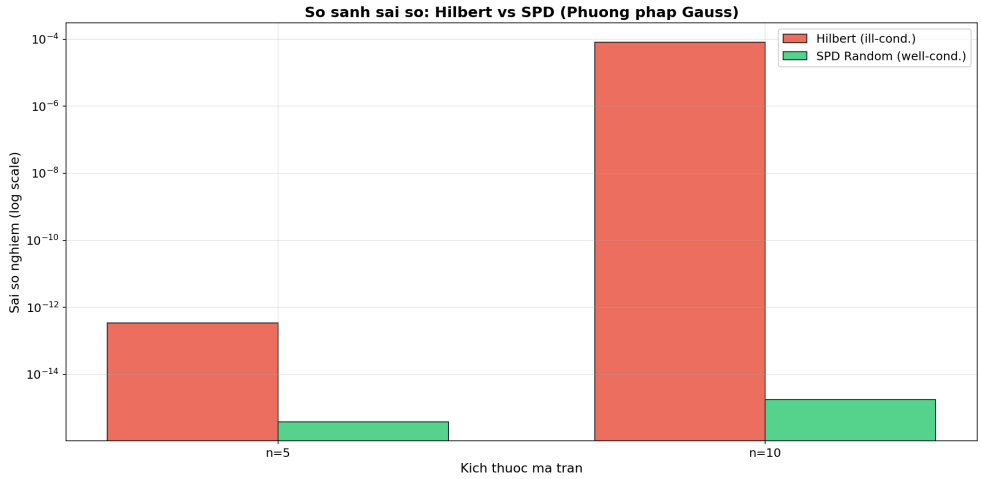


Hình 5 : Ảnh hưởng của số điều kiện $\kappa(A)$ đến sai số nghiệm trên hai lớp dữ liệu: Hilbert và SPD

5.3.4 So sánh Hilbert và SPD: ý nghĩa về ổn định số học

Khi đặt hai nhóm ma trận cạnh nhau, một nhận xét quan trọng có thể rút ra: *độ ổn định của thuật toán không thể tách rời khỏi bản chất của dữ liệu đầu vào*.

Trên ma trận `SPD` ngẫu nhiên, các phương pháp trực tiếp cho kết quả rất ổn định và sai số nhỏ. Trên ma trận `Hilbert`, cùng một thuật toán vẫn có thể cho sai số nghiệm tăng mạnh chỉ vì số điều kiện lớn hơn nhiều. Điều này cho thấy khi đánh giá chất lượng nghiệm trong thực tế, không nên chỉ quan sát một chỉ số duy nhất. Việc theo dõi đồng thời `condition number`, `Relative residual` và `solution error` là cần thiết để tránh kết luận sai rằng bài toán đã được giải tốt chỉ vì residual nhỏ.



Hình 6 : So sánh sai số nghiệm giữa Hilbert và SPD đối với phương pháp Gauss

5.4 Kết luận của phần thực nghiệm

Từ toàn bộ benchmark, có thể nhận thấy không tồn tại một phương pháp tối ưu cho mọi loại dữ liệu.

- *Khử Gauss* và `LU` phù hợp khi cần một lời giải trực tiếp, ổn định và dễ kiểm soát trên các ma trận khả nghịch thông thường.
- `LU` đặc biệt hữu ích nếu cùng một ma trận A được dùng để giải nhiều hệ với các vector b khác nhau.

- `Gauss-Seidel` cho thấy vai trò của phương pháp lặp trong việc khai thác cấu trúc của dữ liệu, nhưng hiệu quả thực tế còn phụ thuộc mạnh vào điều kiện hội tụ, số vòng lặp và chất lượng hiện thực thuật toán.

Quan trọng hơn, kết quả thực nghiệm khẳng định rằng việc lựa chọn thuật toán trong tính toán khoa học không thể chỉ dựa vào công thức lý thuyết, cần xét đồng thời đến *cấu trúc ma trận*, *độ ổn định số học* và *chi phí tính toán quan sát được từ thực nghiệm*. Kết nối các khái niệm như số điều kiện, sai số dư và ổn định số với thực tiễn tính toán thay vì chỉ dừng lại ở lý thuyết trừu tượng.

6 | Kết luận

6.1 Tóm tắt kết quả đạt được

Trong đồ án này, các mục tiêu đề ra về mặt lý thuyết lẫn thực hành đối với các kỹ thuật cốt lõi của đại số tuyến tính số đã được hoàn thành:

- *Mục 3 Phép khử Gauss:* Xây dựng và cài đặt thành công phép khử Gauss với chiến lược chọn phần tử chốt (`Partial Pivoting`) một cách độc lập, ứng dụng vào việc giải hệ phương trình tuyến tính, tính định thức, tìm ma trận nghịch đảo, hạng và xuất cơ sở của các không gian ma trận liên quan.
- *Mục 4 Phân rã ma trận:* Cài đặt thành công kỹ thuật chéo hóa ma trận và phân rã kỳ dị (`SVD`). Sử dụng thư viện `Manim` để xây dựng video minh họa giúp kết nối lý thuyết đại số với hoạt cảnh trực quan.
- *Mục 5 Benchmark:* Thực hiện đánh giá thực nghiệm hoàn chỉnh trên các lớp ma trận ngẫu nhiên (chéo trội, `SPD`, `Hilbert`). Phân tích chi phí tính toán thực tế qua đồ thị log-log và kiểm chứng sự đánh đổi giữa giải thuật trực tiếp (Gauss, `LU`) hay lặp (`Gauss-Seidel`) trước sự biến thiên của số điều kiện (κ).

6.2 Khó khăn gặp phải

Trong quá trình thực hiện đồ án, đã xuất hiện một số thách thức nhất định:

- *Cài đặt thuật toán thuần tuý:* Cài đặt các phép toán trên cấu trúc danh sách lồng nhau của `Python` mà không sử dụng `NumPy` buộc người thực hiện phải đối mặt trực tiếp với các vấn đề về xử lý tốc độ tính toán và cẩn thận quản lý từng phần tử.
- *Xử lý rủi ro số học:* Đối với các lớp ma trận cực kỳ kém ổn định như ma trận `Hilbert`, số điều kiện lớn khiến sai số làm tròn lan truyền đáng kể.
- *Khó khăn về `Manim`:* Việc phải xây dựng các hoạt ảnh toán học phức tạp đòi hỏi phải nắm chắc cả kiến thức lập trình đồ hoạ, tốn nhiều thời gian nghiên cứu tài liệu kỹ thuật.

6.3 Bài học rút ra

Thấy rõ ranh giới lớn giữa *chứng minh hội tụ lý thuyết* và *thực thi an toàn trên máy tính*. Một thuật toán tốt phải cân bằng được giữa chi phí tính toán lý thuyết và khả năng vận hành ổn định trước hiện tượng nhiễu của dấu phẩy động.

Hệ thống quản lý `GitHub` đã được tận dụng tốt cùng với việc tuân thủ các nguyên tắc làm việc chung để tránh xung đột mã nguồn. Có thêm kinh nghiệm trong việt viết mã sạch, chuẩn bị tài liệu mô tả, báo cáo.

7 | Phụ lục

7.1 Tổng hợp các trường hợp kiểm chứng

Bảng 2 : Bảng tổng hợp các trường hợp kiểm chứng thuật toán khử Gauss

Case	Mục đích kiểm tra	Hàm liên quan	Kết luận chính
1	Pivot gần 0	gaussian_eliminate, back_substitution, determinant, inverse	Pivoting hoạt động tốt, nghiệm đúng với Numpy
2	Vô số nghiệm	gaussian_eliminate, rank_and_basis, basis	Rank < n, det = 0, tìm đúng biến tự do
3	Vô nghiệm	gaussian_eliminate	Phát hiện mâu thuẫn
4	Gần suy biến	gaussian_eliminate, verify_solution	Sai số cực thấp

7.2 Thiết lập benchmark

Tóm tắt cấu hình benchmark đã dùng trong Mục 5 nhằm giúp việc đối chiếu số liệu được rõ ràng:

- Thực nghiệm A sử dụng ma trận chéo trội nghiêm ngặt với các kích thước $n \in \{50, 100, 200, 500, 1000\}$.
- thực nghiệm B sử dụng hai lớp dữ liệu:
  1. ma trận `Hilbert` với các kích thước $n \in \{3, 5, 8, 10, 12, 15\}$,
  2. ma trận `SPD` ngẫu nhiên với các kích thước $n \in \{5, 10, 20, 50, 100\}$.
- Với mỗi bài toán, nghiệm chuẩn được chọn là $x_{\text{true}} = (1, 1, ..., 1)^T$ và vế phải được dựng theo công thức $b = Ax_{\text{true}}$.
- Mỗi thuật toán được chạy lặp lại nhiều lần để lấy thời gian thực thi trung bình.
- Các thước đo chính gồm: `Average Runtime`, `Relative Residual Error`, `Solution Error` và `Condition Number`.

7.3 Bảng số liệu benchmark

Bảng 3 : Thời gian thực thi trung bình của ba phương pháp theo kích thước n

Kích thước n	Gauss (s)	LU (s)	Gauss-Seidel (s)
50	0.0074	0.0079	0.0051
100	0.0546	0.0536	0.0183
200	0.4138	0.3867	0.0700
500	6.5889	6.0864	0.4855
1000	66.8685	49.2168	2.1516

Bảng 4 : Sai số tương đối trên ma trận chéo trội theo kích thước n

Kích thước n	Gauss: Relative Residual	LU: Relative Residual	Gauss-Seidel: Relative Residual
50	2.57×10^{-16}	1.80×10^{-16}	1.47×10^{-11}
100	3.99×10^{-16}	2.38×10^{-16}	2.10×10^{-11}
200	6.29×10^{-16}	3.61×10^{-16}	2.24×10^{-11}
500	8.51×10^{-16}	4.84×10^{-16}	4.01×10^{-12}
1000	1.22×10^{-15}	7.14×10^{-16}	4.12×10^{-12}

Bảng 5 : Condition number và sai số nghiệm trên ma trận Hilbert

n	$\kappa(H_n)$	Gauss	LU	Gauss-Seidel
3	$5.24 \cdot 10^{2}$	$8.03 \cdot 10^{-15}$	$6.31 \cdot 10^{-16}$	$2.94 \cdot 10^{-9}$
5	$4.77 \cdot 10^{5}$	$3.49 \cdot 10^{-13}$	$4.08 \cdot 10^{-13}$	$1.56 \cdot 10^{-2}$
8	$1.53 \cdot 10^{10}$	$3.59 \cdot 10^{-8}$	$1.20 \cdot 10^{-7}$	$1.18 \cdot 10^{-2}$
10	$1.60 \cdot 10^{13}$	$8.24 \cdot 10^{-5}$	$7.67 \cdot 10^{-5}$	$1.86 \cdot 10^{-2}$
12	$1.76 \cdot 10^{16}$	0.24	0.22	$1.94 \cdot 10^{-2}$
15	$3.68 \cdot 10^{17}$	inf	inf	$1.48 \cdot 10^{-2}$

Bảng 6 : Condition number và sai số nghiệm trên ma trận SPD ngẫu nhiên

n	$\kappa(A)$	Gauss	LU	Gauss-Seidel
5	8.26	3.88×10^{-16}	3.88×10^{-16}	3.19×10^{-11}
10	26.67	1.89×10^{-15}	2.44×10^{-15}	9.63×10^{-11}
20	101.65	3.64×10^{-15}	3.23×10^{-15}	1.87×10^{-10}
50	624.11	2.83×10^{-14}	3.42×10^{-14}	1.71×10^{-09}
100	2494.19	1.33×10^{-13}	1.29×10^{-13}	0.0505

8 | Tài liệu tham khảo

1. Gilbert Strang. (2023). *Introduction to Linear Algebra* (6th ed.). Wellesley-Cambridge Press.
2. Lloyd N. Trefethen & David Bau III. (1997). *Numerical Linear Algebra*. SIAM.
3. Gene H. Golub & Charles F. Van Loan. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.
4. Cleve Moler. (2004). *Numerical Computing with MATLAB*. SIAM. Truy cập trực tuyến: `https://www.mathworks.com/moler/chapters.html`
5. Manim Community Developers. (2024). *Manim-Mathematical Animation Framework*, v0.18. Truy cập trực tuyến: `https://docs.manim.community`
6. Jake VanderPlas. (2016). *Python Data Science Handbook*. O'Reilly. Truy cập trực tuyến: `https://jakevdp.github.io/PythonDataScienceHandbook/`
7. 3Blue1Brown. (2016). *Essence of Linear Algebra*. YouTube. Truy cập trực tuyến: `https://www.youtube.com/playlist?list=PLZHQObOWTQDPD3MizzM2xVFitgF8hE_ab`
8. Golub, G. H., & Van Loan, C. F. (2013). Matrix Computations (4th ed.). Johns Hopkins University Press.
9. Trefethen, L. N., & Bau III, D. (1997). Numerical Linear Algebra. SIAM.